# ASSIGNMENT – 3

# M.SAIKIRAN

# ROLLNO:24MAB0A17

# MATHEMATICS & COMPUTING

# PROGRAM : 1

Consider the following C program that uses fork() to create a child process:

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
int value = 5;
int main()
{
pid_t pid;
pid = fork();
if (pid == 0) { /* child process */
value += 15;
return 0;
}
        else if (pid> 0) { /* parent process */
wait(NULL);
printf("PARENT: value = %d", value); /* LINE A */
return 0;
}
}
```

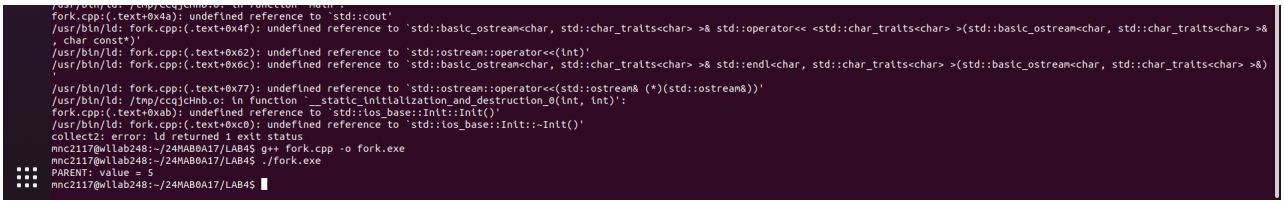
What output will be printed at LINE A when the above program is executed?

**CPP**

```
#include <iostream>
#include <unistd.h>
#include <sys/wait.h>

int value = 5; // Global variable

int main() {
    pid_t pid = fork();

    if (pid == 0) { // Child process
        value += 15;
        return 0;
```

```
    } else if (pid > 0) { // Parent process
        wait(NULL);
        std::cout << "PARENT: value = " << value << std::endl; // LINE A [cite:
18]
    }
    return 0;
}
```

**OUTPUT:**



```
/usr/bin/ld: /tmp/ccqjcHnb.o: in function `main':
fork.cpp:(.text+0x4a): undefined reference to `std::cout'
/usr/bin/ld: fork.cpp:(.text+0x4f): undefined reference to `std::basic_ostream<char, std::char_traits<char> >& std::operator<< <std::char_traits<char> >(std::basic_ostream<char, std::char_traits<char> >&
, char const*)'
/usr/bin/ld: fork.cpp:(.text+0x62): undefined reference to `std::ostream::operator<<(int)'
/usr/bin/ld: fork.cpp:(.text+0x6c): undefined reference to `std::basic_ostream<char, std::char_traits<char> >& std::endl<char, std::char_traits<char> >(std::basic_ostream<char, std::char_traits<char> >&)
'
/usr/bin/ld: fork.cpp:(.text+0x77): undefined reference to `std::ostream::operator<<(std::ostream& (*)(std::ostream&))'
/usr/bin/ld: /tmp/ccqjcHnb.o: in function `__static_initialization_and_destruction_0(int, int)':
fork.cpp:(.text+0xab): undefined reference to `std::ios_base::Init::Init()'
/usr/bin/ld: fork.cpp:(.text+0xc0): undefined reference to `std::ios_base::Init::~Init()'
collect2: error: ld returned 1 exit status
mnc2117@wllab248:~/24MAB0A17/LAB4$ g++ fork.cpp -o fork.exe
mnc2117@wllab248:~/24MAB0A17/LAB4$ ./fork.exe
PARENT: value = 5
mnc2117@wllab248:~/24MAB0A17/LAB4$
```

**OUTPUT : PARENT: value = 5**

**EXPLANATION:**

When a process calls `fork()`, the operating system creates a **separate copy** of the address space for the child. While the child process modifies its own copy of the variable `value` (changing it to 20), the parent's copy remains unchanged. Because they do not share memory, the parent prints the original value.

# PROGRAM:2

Consider the following C program:

```
#include <stdio.h>
#include <unistd.h>
int main()
{
int i;
for (i = 0; i< 4; i++)
fork();
return 0;
}
```

How many total processes are created when this program runs?
(Including the initial parent process, how many processes are created by the program)

**CPP**

```
#include <iostream>
#include <unistd.h>
using namespace std;
int main() {
    for (int i = 0; i < 4; i++)
{
        fork();
        cout<<"process"<<i<<endl;
}
    // Each process will reach this point
    return 0;
}
```

**OUTPUT:**

Output: 16 total processes.

**EXPLANATION:**

Explanation: Each time fork() is called in a loop, the number of processes doubles.

# PROGRAM:3

Consider the following C program that uses fork() and execlp():

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
int main()
{
pid_t pid;
pid = fork();
if (pid < 0) {
        fprintf(stderr, "Fork Failed");
return 1;
}
else if (pid == 0) {
execlp("/bin/ls","ls", NULL);
```

```
printf("LINE J");
}
else {
wait(NULL);
printf("Child Complete");
}
return 0;
}
```

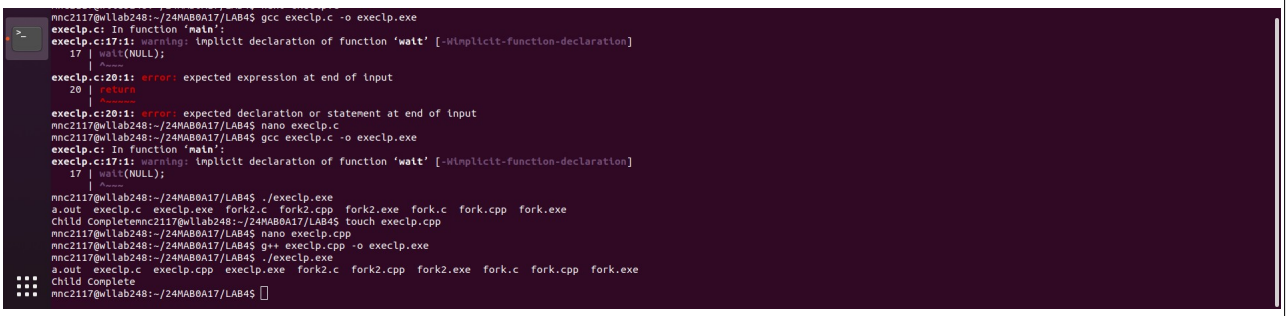
a) Will the line LINE J be printed? Explain your answer based on how execlp() works.

b) Explain the circumstances under which(Conditions) the line of code marked printf("LINE J") will be reached.

**CPP**

```
#include <iostream>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        execlp("/bin/ls", "ls", NULL);
        std::cout << "LINE J" << std::endl; // Only runs if execlp fails
    } else if (pid > 0) {
        wait(NULL);
        std::cout << "Child Complete" << std::endl;
    }
    return 0;
}
```

**OUTPUT:**



```
mnc2117@wllab248:~/24MAB0A17/LAB4$ gcc execlp.c -o execlp.exe
execlp.c: In function ‘main’:
execlp.c:17:1: warning: implicit declaration of function ‘wait’ [-Wimplicit-function-declaration]
   17 | wait(NULL);
      | ^~~~
execlp.c:20:1: error: expected expression at end of input
   20 | return
      | ^~~~~~
execlp.c:20:1: error: expected declaration or statement at end of input
mnc2117@wllab248:~/24MAB0A17/LAB4$ nano execlp.c
mnc2117@wllab248:~/24MAB0A17/LAB4$ gcc execlp.c -o execlp.exe
execlp.c: In function ‘main’:
execlp.c:17:1: warning: implicit declaration of function ‘wait’ [-Wimplicit-function-declaration]
   17 | wait(NULL);
      | ^~~~
mnc2117@wllab248:~/24MAB0A17/LAB4$ ./execlp.exe
a.out  execlp.c  execlp.exe  fork2.c  fork2.cpp  fork2.exe  fork.c  fork.cpp  fork.exe
Child Completemnc2117@wllab248:~/24MAB0A17/LAB4$ touch execlp.cpp
mnc2117@wllab248:~/24MAB0A17/LAB4$ nano execlp.cpp
mnc2117@wllab248:~/24MAB0A17/LAB4$ g++ execlp.cpp -o execlp.exe
mnc2117@wllab248:~/24MAB0A17/LAB4$ ./execlp.exe
a.out  execlp.c  execlp.cpp  execlp.exe  fork2.c  fork2.cpp  fork2.exe  fork.c  fork.cpp  fork.exe
Child Complete
mnc2117@wllab248:~/24MAB0A17/LAB4$
```

**a)** Will LINE J be printed? **No.** Explanation: The `execlp()` system call replaces the current process image with a new process image (in this case, the `ls` command). If `execlp()` is successful, the entire text segment of the child process is overwritten, and the original program code following the `execlp()` call is "gone." Therefore, the `printf` at LINE J is never executed.

**b) Conditions to reach LINE J:** LINE J will only be reached if the **`execlp()` call fails**. This could happen if:

- The path to the executable is incorrect.
- The system lacks the resources to load the new image.
- The user does not have permission to execute the file.

# PROGRAM:4

Consider the following C program that creates a child process using fork():

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
int main()
{

pid_t pid, pid1;
/* fork a child process */
pid = fork();
if (pid < 0) { /* error occurred */
fprintf(stderr, "Fork Failed");
return 1;
}
else if (pid == 0) { /* child process */
pid1 = getpid();
printf("child: pid = %d\n", pid); // LINE A
```

```
        printf("child: pid1 = %d\n", pid1); // LINE B
        }
        else { /* parent process */
        pid1 = getpid();
        printf("parent: pid = %d\n", pid); // LINE C
        printf("parent: pid1 = %d\n", pid1); // LINE D
        wait(NULL);
                }
        return 0;}
```

Assume the parent process has PID 2600 and the child process has PID 2603. What values will
be printed at:

- LINE A
- LINE B
- LINE C
- LINE D

**CPP**

```
        #include <iostream>
        #include <unistd.h>
        #include <sys/types.h>
        #include <sys/wait.h>

        int main() {
            pid_t pid, pid1;

            /* fork a child process */
            pid = fork();

            if (pid < 0) {
                /* error occurred */

          std::cerr << "Fork Failed" << std::endl;
                return 1;
            }
    else if (pid == 0) {
                /* child process */
                pid1 = getpid();
                std::cout << "child: pid = " << pid << std::endl;   // LINE A
                std::cout << "child: pid1 = " << pid1 << std::endl; // LINE B
            }
            else {
```

```
/* parent process */
        pid1 = getpid();
        std::cout << "parent: pid = " << pid << std::endl;   // LINE C
        std::cout << "parent: pid1 = " << pid1 << std::endl; // LINE D
wait(NULL);
    }

    return 0;
  }
```

**OUTPUT:**

```
mnc2117@wllab248:~/24MAB0A17/LAB4$ touch fork4.c
mnc2117@wllab248:~/24MAB0A17/LAB4$ nano fork4.c
mnc2117@wllab248:~/24MAB0A17/LAB4$ gcc fork4.c -o fork4.exe
fork4.c: In function 'main':
fork4.c:23:1: warning: implicit declaration of function 'wait' [-Wimplicit-function-declaration]
   23 | wait(NULL);
      | ^~~~
mnc2117@wllab248:~/24MAB0A17/LAB4$ ./fork4.exe
parent: pid = 6337
parent: pid1 = 6336
child: pid = 0
child: pid1 = 6337
mnc2117@wllab248:~/24MAB0A17/LAB4$ touch fork4.cpp
mnc2117@wllab248:~/24MAB0A17/LAB4$ nano fork4.cpp


Use "fg" to return to nano.

[1]+  Stopped                 nano fork4.cpp
mnc2117@wllab248:~/24MAB0A17/LAB4$ nano fork4.cpp
mnc2117@wllab248:~/24MAB0A17/LAB4$ nano fork4.cpp
mnc2117@wllab248:~/24MAB0A17/LAB4$ g++ fork4.cpp -o fork4.exe
mnc2117@wllab248:~/24MAB0A17/LAB4$ ./fork4.exe
parent: pid = 6805
parent: pid1 = 6804
child: pid = 0
child: pid1 = 6805
mnc2117@wllab248:~/24MAB0A17/LAB4$
```

**Explanation:** `getpid()` returns the ID of the caller, while `fork()` returns the child's ID to the parent and `0` to the child .

**Assumed:** Parent PID = 2600, Child PID = 2603.

**Output:**

**LINE A (child's `pid` variable):** `0`

- **LINE B (child's `pid1` variable):** `2603`
- **LINE C (parent's `pid` variable):** `2603`
- **LINE D (parent's `pid1` variable):** `2600`

# PROGRAM:5

Consider the following C program that creates a child process using fork():

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
#define SIZE 5
int nums[SIZE] = {0, 1, 2, 3, 4};
int main()

{
int i;
pid_t pid;
pid = fork();
if (pid == 0) { // Child process
for (i = 0; i< SIZE; i++) {
nums[i] *= -i;
printf("CHILD: %d ", nums[i]); // LINE X
}
}
else if (pid > 0) { // Parent process
wait(NULL);
for (i = 0; i< SIZE; i++)
printf("PARENT: %d ", nums[i]); // LINE Y
}
return 0;
}
```

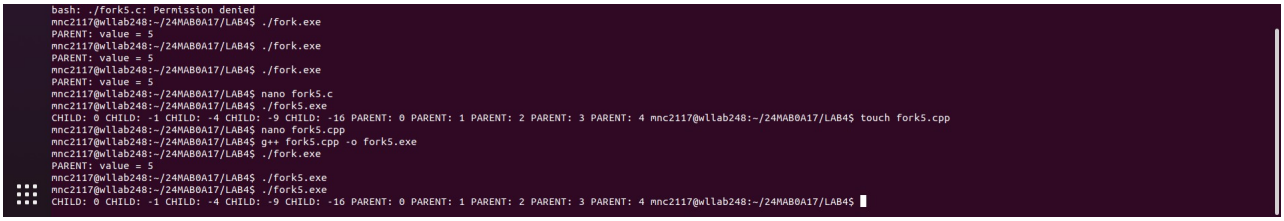
What values will be printed at LINE X and LINE Y by the child process? Show the output.

**CPP:**

```
#include <iostream>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int nums[5] = {0, 1, 2, 3, 4};
    pid_t pid = fork();
```

```
                if (pid == 0) { // Child
    for (int i = 0; i < 5; i++) {
            nums[i] *= -i;
            std::cout << "CHILD: " << nums[i] << " "; // LINE X
        }
    } else if (pid > 0) { // Parent
        wait(NULL);
        for (int i = 0; i < 5; i++)
            std::cout << "PARENT: " << nums[i] << " "; // LINE Y
    }
    return 0;
}
```

**OUTPUT**



```
bash: ./fork5.c: Permission denied
mnc2117@wllab248:~/24MAB0A17/LAB4$ ./fork.exe
PARENT: value = 5
mnc2117@wllab248:~/24MAB0A17/LAB4$ ./fork.exe
PARENT: value = 5
mnc2117@wllab248:~/24MAB0A17/LAB4$ ./fork.exe
PARENT: value = 5
mnc2117@wllab248:~/24MAB0A17/LAB4$ nano fork5.c
mnc2117@wllab248:~/24MAB0A17/LAB4$ ./fork5.exe
CHILD: 0 CHILD: -1 CHILD: -4 CHILD: -9 CHILD: -16 PARENT: 0 PARENT: 1 PARENT: 2 PARENT: 3 PARENT: 4 mnc2117@wllab248:~/24MAB0A17/LAB4$ touch fork5.cpp
mnc2117@wllab248:~/24MAB0A17/LAB4$ nano fork5.cpp
mnc2117@wllab248:~/24MAB0A17/LAB4$ g++ fork5.cpp -o fork5.exe
mnc2117@wllab248:~/24MAB0A17/LAB4$ ./fork.exe
PARENT: value = 5
mnc2117@wllab248:~/24MAB0A17/LAB4$ ./fork5.exe
mnc2117@wllab248:~/24MAB0A17/LAB4$ ./fork5.exe
CHILD: 0 CHILD: -1 CHILD: -4 CHILD: -9 CHILD: -16 PARENT: 0 PARENT: 1 PARENT: 2 PARENT: 3 PARENT: 4 mnc2117@wllab248:~/24MAB0A17/LAB4$
```

**Explanation:** Similar to Question 1, the child modifies its own copy of the `nums` array nums = {0,1,2,3,4}.

**Output:**

**Output at LINE X (Child):** 0 -1 -4 -9 -16

**Output at LINE Y (Parent):** 0 1 2 3 4

**Reasoning:** Even though the child modifies its array, it is operating on a **copy**. When the parent resumes after the `wait(NULL)`, its original `nums` array remains exactly as it was initialized.